

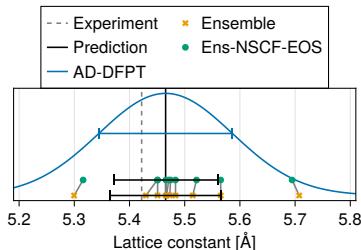
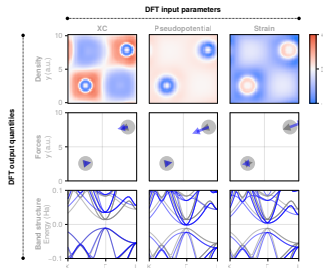
Tutorial: UQ for DFT

Algorithmic differentiation in DFTK.jl¹

Niklas F. Schmitz

Mathematics for Materials Modelling (matmat.org), EPFL

CECAM-HQ-EPFL, Nov 25, 2025



¹N. F. Schmitz, B. Ploumhans, M. F. Herbst. *npj Comput. Mater.*, (accepted: 17 Nov 2025). arXiv:2509.07785.

Uncertainty due to what? Parameters in DFT

- Model (exchange-correlation, pseudopotentials, ...)
- Numerics (basis E_{cut} , k-points, SCF tolerance, ...)

Tasks with gradients

- structure optimization, ...
- learning exchange-correlation, ...
- uncertainty propagation

This tutorial, new tool: Algorithmic Differentiation for plane-wave DFT (AD-DFPT)^a

^aN. F. Schmitz, B. Ploumhans, M. F. Herbst. *npj Comput. Mater.*, (accepted: 17 Nov 2025). arXiv:2509.07785.

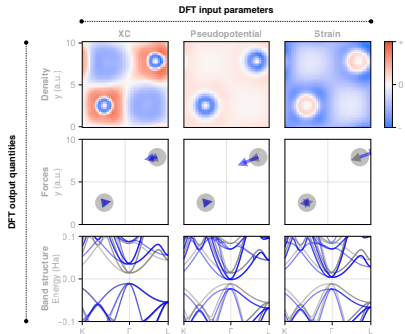
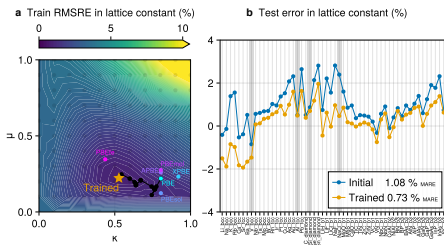


Figure: Derivatives from AD-DFPT in DFTK.jl (dftk.org)

Learning exchange-correlation parameters²



- ▶ Testing on Sol58LC dataset, Training on $C = \text{Si, Al, V, NaCl}$
- ▶ Optimize PBE exchange (2 parameters θ) to expt. lattice const.

$$L_{\text{xc}}(\theta) = \frac{1}{N} \sum_{i=1}^N \left(\frac{a^*(C_i, \theta) - a_i^{\text{expt}}}{a_i^{\text{expt}}} \right)^2, \quad a^*(C, \theta) = \arg \min_a \min_P \mathcal{E}(C, a, \theta, P)$$

- ▶ Practical challenge to compute $\frac{\partial L_{\text{xc}}}{\partial \theta}$:
 - ▶ **Nested iterative methods** (eigensolver, SCF, lattice optimization)
 - ▶ Unusual derivative $\frac{\partial a^*}{\partial \theta} = - \left(\frac{\partial^2 E}{\partial a^2} \right)^{-1} \frac{\partial^2 E}{\partial \theta \partial a}$
- ▶ Fully self-consistent

²N. F. Schmitz, B. Ploumhans, M. F. Herbst. *npj Comput. Mater.*, (accepted: 17 Nov 2025). arXiv:2509.07785.

Forward uncertainty quantification³

- ▶ Given parameter uncertainty $\theta \sim \mathcal{N}(\theta_0, \Sigma)$, propagate to lattice constant $a^*(\theta)$

- ▶ Linearization around mean θ_0 :

$$a^*(\theta) \approx a^*(\theta_0) + J(\theta - \theta_0)$$

$$\implies \mathcal{N}(a^*(\theta_0), J\Sigma J^\top) \text{ with } J = \left. \frac{\partial a^*}{\partial \theta} \right|_{\theta_0} \text{ from AD-DFPT}$$

- ▶ only 1 relaxation + 1 derivative, no N -fold ensemble required
- ▶ general

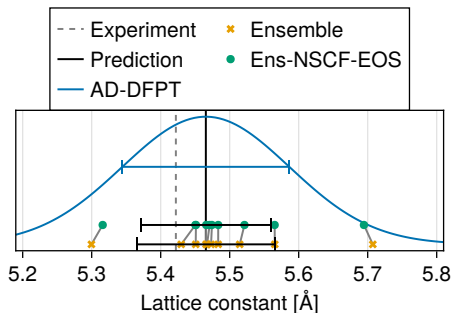
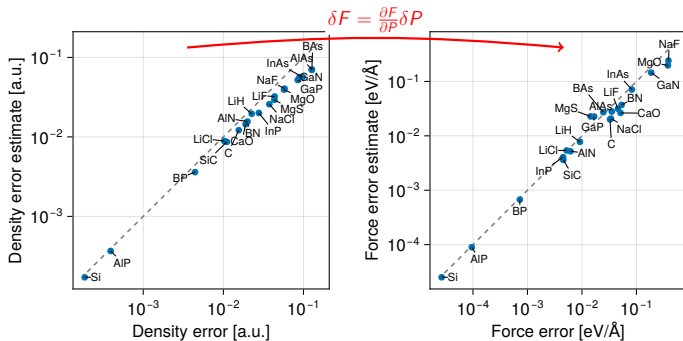


Figure: XC uncertainty (here BEEF^a) propagated to lattice constant on Si (AD-DFPT and Ensemble baselines)

^aMortensen, Kaasbjerg, Frederiksen, Nørskov, Sethna, Jacobsen. (2005). Phys. Rev. Lett., 95, 216401.

³N. F. Schmitz, B. Ploumhans, M. F. Herbst. *npj Comput. Mater.*, (accepted: 17 Nov 2025). arXiv:2509.07785.

Plane-wave error estimation⁵



Setup: Use a uniform low $E_{\text{cut}} = 20$ Ha across solids.

Perturbative density error estimates δP following method of⁴.

⇒ propagate to quantities of interest by AD (here forces).

⁴Cancès, Dusson, Kemlin, Levitt. SIAM J. Sci. Comp., 44, B1312 (2022)

⁵N. F. Schmitz, B. Ploumhans, M. F. Herbst. *npj Comput. Mater.*, (accepted: 17 Nov 2025). arXiv:2509.07785.

Algorithmic differentiation (AD)

AD: automatic derivatives of computer programs by applying chain rule to primitive operations. $+$, $*$, `matmul`, ...

AD systems: Pytorch, JAX, Enzyme, ForwardDiff.jl, ...

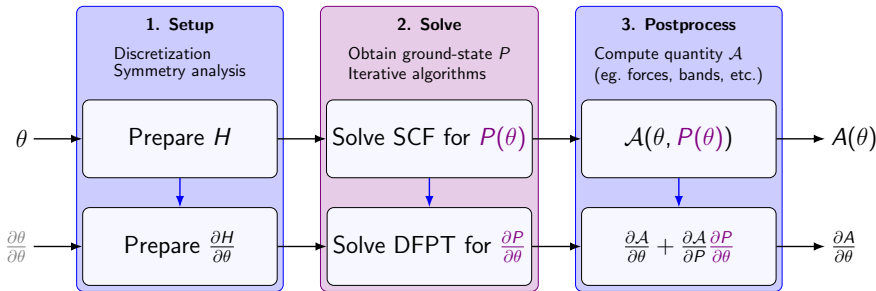
Two basic modes:

- ▶ Forward: sends input tangents v as $(x, v) \mapsto (f(x), Df(x)v)$
- ▶ Reverse: sends output tangents w to $Df(x)^\top w$
("backprop"; not yet in DFTK)

Key tool: **custom AD rules** to define new primitives

- ▶ implicit differentiation for numerical solvers $\implies$ **AD-DFPT**
- ▶ foreign language calls (e.g. FFTW in C)
- ▶ symmetry considerations

End-to-end DFT derivatives with AD-DFT



- ▶ Setup and postprocessing handled by **general-purpose AD** system (ForwardDiff.jl)
- ▶ We **impose custom AD rule**: Converge SCF, then DFPT solver⁶.

⁶Cancès, Herbst, Kemlin, Levitt, Stamm. Lett. in Math. Phys. 113, 21 (2023).

DEMO

[https://github.com/niklasschmitz/
tutorial-cecam-workshop-dftk-2025](https://github.com/niklasschmitz/tutorial-cecam-workshop-dftk-2025)

Summary

- ▶ We introduced AD-DFPT
 - ▶ embeds DFPT into general-purpose AD system
 - ▶ use cases: Optimization, Learning, Uncertainty propagation, ...
 - ▶ Forward-mode scales with number of input parameters (just like DFPT)
- ▶ Future directions:
 - ▶ UQ on more properties (phonons, electric fields)
 - ▶ UQ calibration against experiment (or higher)
 - ▶ self-consistent ML-XC training on properties (use UQ regularizers)
 - ▶ extend implementation to mGGA & nonlocal XC, beyond GGA
 - ▶ extend to reverse-mode for $O(1)$ gradients in high dimensions

Thank you! Questions?

- ▶ N. F. Schmitz, B. Ploumhans, M. F. Herbst. *npj Comput. Mater.* (accepted: 17 Nov 2025). arXiv:2509.07785.
- ▶ All examples: <https://github.com/niklasschmitz/ad-dfpt>
- ▶ Framework: DFTK.jl (dftk.org)

Appendix

Motivation: Derivatives in DFT

- Physical responses $\longleftrightarrow$ derivatives
 - Forces, stresses, phonons, dielectric response, ...
- Parameter optimization by gradients
 - inverse design of structure, ...
 - learning exchange-correlation, ...

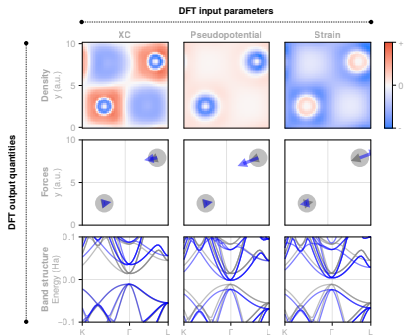


Figure: Computed with AD-DFPT in DFTK.jl (dftk.org)

Kohn-Sham Density Functional Theory (DFT)

Parameters θ (positions, lattice, model parameters,...)

Ground state: Find $P(\theta)$ minimizing energy $\mathcal{E}(\theta, P)$.

Self-consistent field (SCF) equations:

$$\begin{cases} H(\theta, P)\psi_n = \varepsilon_n\psi_n, & n = 1, \dots, N \\ P = \sum_{n=1}^N f(\varepsilon_n)\psi_n\psi_n^\dagger, \end{cases}$$

or compactly, writing f as a matrix function:

$$P = f(H(\theta, P)).$$

Plane-wave DFT: Large basis size N_b (avoid $\mathcal{O}(N_b^2)$ at all cost)

$\implies$ matrix-free solvers working only on N orbitals are standard here

Density Functional Perturbation Theory (DFPT)

- ▶ Well-established method for DFT derivatives⁷
- ▶ Differentiate SCF equations at solution P :

$$P = f(H(\theta, P)) \iff \frac{\partial P}{\partial \theta} = (1 - \chi_0 K)^{-1} \chi_0 \frac{\partial H}{\partial \theta},$$

where

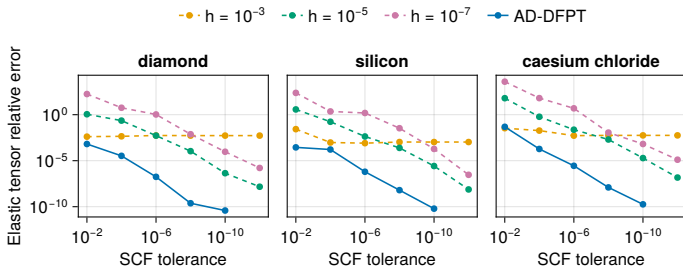
$\chi_0 := \frac{\partial f}{\partial H}$ the independent susceptibility of the density matrix,

$K := \frac{\partial H}{\partial P}$ due to nonlinear Hartree and XC terms.

- ▶ Solve for $\frac{\partial P}{\partial \theta}$ (efficient iterative solvers on N orbitals, similar to SCF)

⁷Baroni, Gironcoli, Dal Corso, Giannozzi (2001). Phonons and related crystal properties from density-functional perturbation theory. Rev. Mod. Phys., 73, 515–562.

Elasticity



- ▶ Accuracy better than finite differences, even at loose SCF tolerance
- ▶ Implementation simply by defining⁸ (strain η)
 - ▶ Stress $\sigma(\eta) = \frac{1}{V(\eta)} \frac{\partial \mathcal{E}}{\partial \eta}$, “Hellmann-Feynman Thm.”
 - ▶ Elastic tensor $C = \frac{\partial \sigma}{\partial \eta}|_{\eta^*}$, AD-DFPT
- ▶ AD-DFPT combines precision and simplicity.

⁸https://docs.dftk.org/stable/examples/elastic_constants/

Inverse design: Band gap by strain

a

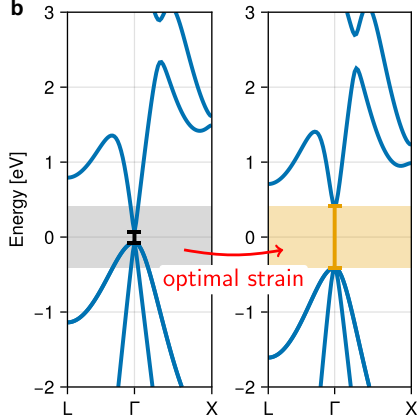
```
using DFTK, PseudoPotentialData, AtomsIO
using ForwardDiff, DifferentiationInterface, Optim

system = load_system("mp-2534-GaAs.cif")
pseudopotentials = PseudoFamily("dojo.nc.sr.pbe.v0_4_1.standard.upf")
model0 = model_DFT(system; functionals=PBE(), pseudopotentials,
    smearing=Smeearing.Gaussian(), temperature=1e-3)

function strain_bandgap( $\eta$ )
    model = Model(model0; lattice=(1 +  $\eta$ ) * model0.lattice)
    basis = PlaneWaveBasis(model; Ecut=42, kgrid=(8, 8, 8))
    scfres = self_consistent_field(basis; tol=1e-6)
    eigenvalues_I = scfres.eigenvalues[1]
     $\epsilon_{\text{vbm}}$  = maximum(eigenvalues_I[eigenvalues_I . $\leq$  scfres. $\epsilon_F$ ])
     $\epsilon_{\text{cbm}}$  = minimum(eigenvalues_I[eigenvalues_I . $>$  scfres. $\epsilon_F$ ])
     $\epsilon_{\text{cbm}}$  -  $\epsilon_{\text{vbm}}$ 
end

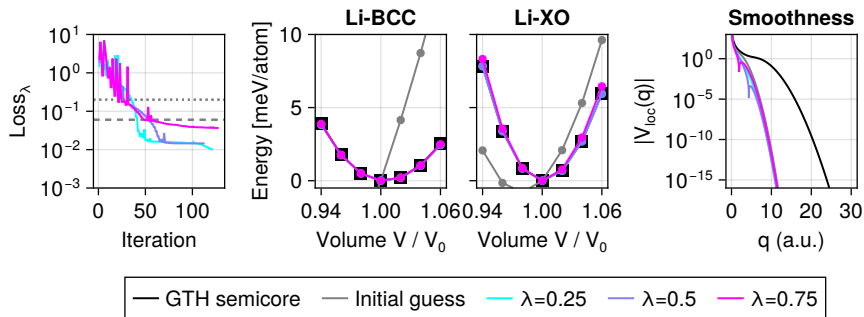
 $\eta_0$  = [0.0] # Initialize with zero strain (at equilibrium)
bandgap_target = 0.03 # Target band gap in [Ha]
bandgap_loss( $\eta$ ) = (bandgap_target - strain_bandgap( $\eta$ ))2
res = Optim.optimize(bandgap_loss,  $\eta_0$ , BFGS(),
    Optim.Options(; iterations=5, x_abstol=1e-3);
    autodiff=AutoForwardDiff())
println("Optimized design strain  $\eta$  = ", res.minimizer)
```

b

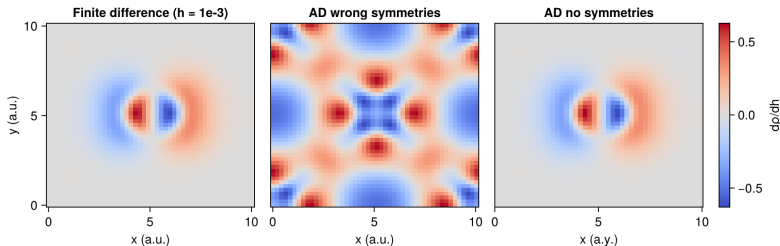


- Easy: This is a full code example, just write a loss function with DFT
- `Optim.optimize` requests AD gradients, which triggers AD-DFPT

Optimizing pseudopotentials



Careful: AD and symmetry-breaking perturbations



A symmetry-breaking density derivative $\frac{\partial \rho}{\partial \theta}$ computed in three ways.

- ▶ The problem: Our automated symmetry detection initially only considered the unperturbed crystal structure
- ▶ The fallback solution: Disable symmetries (expensive) or handle manually
- ▶ Better: determine stabilizer subgroup for perturbation automatically inside custom AD rule